Report on Bayesian Inference, MCMC and KDE

January 20, 2021

1 Bayesian Inference

Consider a forward model $F : \mathbb{R}^m \to \mathbb{R}^n, x \mapsto y$. The coresponding inverse problem is to determine unknown parameters x^* given measurements y^*.

Such question can be interpreted in probability language. It is natural to consider posterior distribution density $p_{X|Y}(x|y = y^*)$. A candidate parameter $\hat{x}$ may be the maximizer, or the expectation of the posterior. We can also use the variance to indicate how good our solution is. Suppose we want to find the maximizer. By Bayes' Theorem

$$p_{X|Y}(x|y = y^*) = \frac{p_{Y|X}(y^*|x)p_X(x)}{p_Y(y^*)},$$

it suffices to maximize $p_{Y|X}(y^*|x)p_X(x)$.

Marginal density p_Y is sometimes difficult to get. For example, when measuring one's height x, without knowing x, we do not know how measurement y distributes. But we know the conditional distribution of y given x is a normal distribution with mean x. Although p_Y is hard to get, it does not matter. Here $p_Y(y)$ is a constant; in MCMC algorithm, only a function proportional to target density is demanded; and in Metropolis-adjusted Langevin algorithm, such constant is either incorporated into step-size or eliminated after taking log and differentiating.

Often priori $p_X(x) \propto e^{-\lambda\psi(x)}$. ψ varies according to practices. Noise model may be Gaussian, i.e. $p_{Y|X}(y|x) \propto e^{-\frac{1}{2\sigma^2}\|F(x)-y\|^2}$. Then

$$-\log(p_{Y|X}p_X) = \frac{1}{2\sigma^2}\|F(x) - y\|^2 + \lambda\psi(x).$$

This coincides with a Tikhonov minimizer.

2 Metropolis-Hastings Algorithm

2.1 Fundamentals

When it comes to computing summerizing statistics, such as mean and covariance, MCMC algotithm distinguishes itself as a good method, especially in high dimensional cases. Let $p_\xi(x)$ be the target density, and $f(x)$ be an arbitrary function. Suppose $\{x_i\}_{1\leq i\leq N}$ is a set of i.i.d. samples from target p_ξ, then by Central Limit Theorem

$$\frac{\sum_{i=1}^N f(x_i) - N\cdot\mu}{\sqrt{N\cdot Var[f(\xi)]}} \xrightarrow{d} N(0,1),$$

we have

$$P(a < \frac{\sqrt{N}}{\sqrt{Var[f(\xi)]}} e_N < b) \longrightarrow \Psi(b) - \Psi(a), \tag{1}$$

where error $e_N = \frac{1}{N}\sum_{i=1}^N f(x_i) - \int_{\mathbb{R}} f(x)p(x)dx$ and Ψ is the standard normal distribution. Hence the integral is approximated by the sum of f of samples, with error $O(\frac{1}{\sqrt{N}})$. Now the remaining problem is sampling.

A morkov chain is a sequence of random variables ξ_n (states) s.t. the succeeding state is dependent only on current state. In the case where the state space is continuous, densties may be given by integral transform

$$p_{\xi_{n+1}}(y) = \int p(y,x)p_{\xi_n}(x)dx. \tag{2}$$

The kenel $p(y,x)$ is said to be a transition. For a discrete state space, $p_j^{(n+1)} = \sum_i p_{j,i}p_i^{(n)}$. Under certain conditions, there exists a unique equilibrium distribution. (See appendix.) Roughly speaking, Metropolis-Hastings algorithm abtains samples by generating a Markov Chain with target equilibrium distribution. Let $g(y,x)$ be a proposal distribution and f proportional to target p.

The acceptance ratio $\alpha = \frac{f(x')}{f(x_i)}\frac{g(x_i,x')}{g(x',x_i)} = \frac{p(x',x)}{p(x,x')} = \frac{\textit{the likelihood of jumping from } x' \textit{ to } x}{\textit{the likelihood of jumping from } x \textit{ to } x'}$. If the probability of jumping from x to x' is relatively higher, i.e., $\alpha > 1$, then x' is surely accepted. If it is relatively lower, we still give it a chance by camparing it with the uniform distribution. In general, we tend to obtain samples of high probability, but still visit those of low probability occasionally. Note that the mean and covariance of the proposal density characterize jump region and jump width. In general, it is not clear which proposal g we should choose.

Algorithm 1 Metropolis-Hastings algorithm

Require: Input f, initial x_1 and N;
  for i=1: N-1 **do**
    Sample x' from proposal $g(x, x_i)$
    Compute acceptance ratio $\alpha = \frac{f(x')}{f(x_i)} \frac{g(x_i,x')}{g(x',x_i)}$
    Generate $u \sim U(0, 1)$
    if $\alpha > u$ **then**
      $x_{i+1} = x'$;
    else
      $x_{i+1} = x_i$
    end if
  end for

2.2 Experiments

Run Metropolis-Hastings algotithm in Matlab. First use Gaussian ditribution as proposal, with mean value being current sample, and abandon the first half of samples. Tune the jump width by ajusting the variance. Define A.R. =Acceptance Rate=$\frac{\#\ accepted\ proposal\ samples}{\#\ all\ samples}$. To compare samples with target density, use

$$p(t_i) \cdot |\Delta t_i| \approx \frac{\#\ samples\ falling\ into\ \Delta t_i}{\#\ all\ samples}$$

to estimate $p(t_i)$.

Take Gaussian as target density. Experience suggests that a sampling with a jump width s.t. $A.R. \approx 0.50$ may have better effects. See Figure 1, 2 and 3.

We can also change the proposal to exponential distribution. Its mean value is 0, and jump width is characterized by the parameter λ. See Figure 4. (A.R. = Acceptance Rate; J.W. = Jump Width.)

For high-dimensional cases, Gibbs Sampling becomes a promising alternative. Take multi-normal distribution of two variables as target, and update every components one by one. Figure 5-10 illustrates sampling's dependence on jump width and acceptance rate.

In the 1-D case, let $p \propto e^{-\frac{x^2}{2}}$, and Gaussian be proposal distribution. Employ MCMC to compute $\int \frac{1}{\sqrt{2\pi}} x^2 e^{-\frac{1}{2}x^2} dx = 1$. Relative error seems random with respect to acceptance rate. (See figure 17.)

3 Metropolis-adjusted Langevin Algorithm

Still, let $p(x)$ denote the target density, where $x \in \mathbb{R}^d$. (The Langevin Part.) While (2) uses integral transform, here we employ Langevin Itô diffusion to generate a Markov chain (of continuous time):

$$\dot{x} = \nabla \log p(x) + \sqrt{2}\dot{w},$$

where w is a standard Brownian motion. (I do not know how) but it can be proved that as time $t \to \infty$, the distribution of $x(t)$ approaches p.

One of the sampling methods of the Langevin diffusion is given by the update rule

$$x'_{n+1} = x_n + \tau \nabla \log p(x_n) + \sqrt{2\tau}\xi_n,$$

where $\tau > 0$ is a fixed step size, and ξ_k is a multivariate normal distribution with mean 0 and covariance matrix equal to identity. (The Metropolis Part.) Thereafter x'_{n+1} is accepted as the next state or not by comparing its acceptance ratio with a uniform variable. Note that $\nabla \log p = \nabla \log f$ if $f \propto p$.

Note that x'_{n+1} is actually a multivariable normal distribution with mean $x_n + \tau \nabla \log p(x_k)$ and covariance matrix equal to $2\tau I$. So the proposal density $g(y, x) \propto e^{-\frac{1}{4\tau}\|y-(x+\tau \nabla \log p(x))\|^2}$. Hence Hastings-Adjusted Langevin is a special MCMC. If the step size is tuned well, Metrololis-adjusted Langevin can also realize MAP.

Algorithm 2 Metropolis-adjusted Langevin algorithm

Require: Input f, initial x_1, N, and step size τ;
  for i=1: N-1 **do**
    Sample $x' \sim x_i + \tau \nabla \log p(x_i) + \sqrt{2\tau}\xi$
    Compute acceptance ratio $\alpha = \frac{f(x')}{f(x_i)} \frac{g(x_i, x')}{g(x', x_i)}$
    Generate $u \sim U(0, 1)$
    if $\alpha > u$ **then**
      $x_{i+1} = x'$;
    else
      $x_{i+1} = x_i$
    end if
  end for

Figure 11-16 illustrate effects of different step sizes.

4 Kernel Density Estimation

Suppose we have an unknown target density $f(x)$, and a set of samples $\{x_i\}_{i=1}^n$ drawn from f. Now we want to estimate f by the samples. The kernel density estimator is

$$\hat{f}_h(x) = \frac{1}{n}\sum_{i=1}^{n} K_h(x - x_i) = \frac{1}{nh}\sum_{i=1}^{n} K(\frac{x - x_i}{h}),$$

where kernel K is a non-negative density, $K_h(x) = \frac{1}{h}K(\frac{x}{h})$ and $h > 0$ is the bandwidth. The smaller the bandwidth, the sharper the K_h. See Figure 18.

With h being too big, the estimator may be too flat, hence not accurate. With h being too small, the estimator may vibrate a lot. See Figure 19. There are also algorithms that adjust h automatically, according to the density around the test point (Variable KDE). One common bandwidth choice is

$$h_i = \frac{C}{[n \cdot \hat{p}(x_i)]^d}, \quad \hat{h}(x) = \frac{1}{n}\sum_{i=1}^{n} K_{h_i}(x),$$

where C is a constant, $\hat{p}$ is a rough estimate of p given by frequencies of samples, and d is the dimension of samples. See Figure 20.

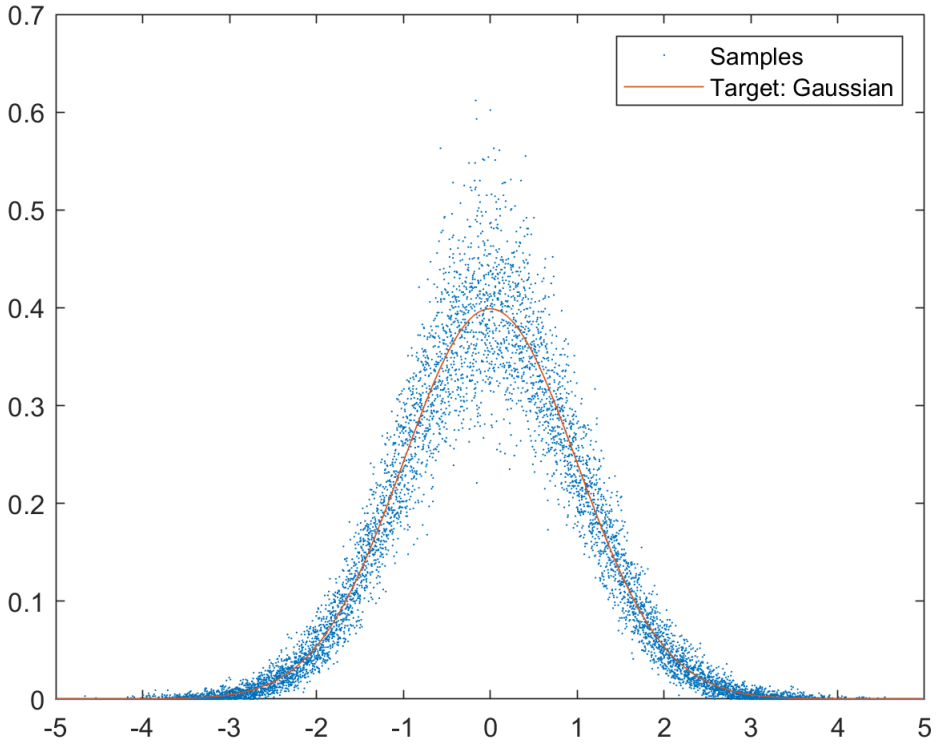


Figure 1: 50 Thousand Samples; Acceptance Rate = 0.1212

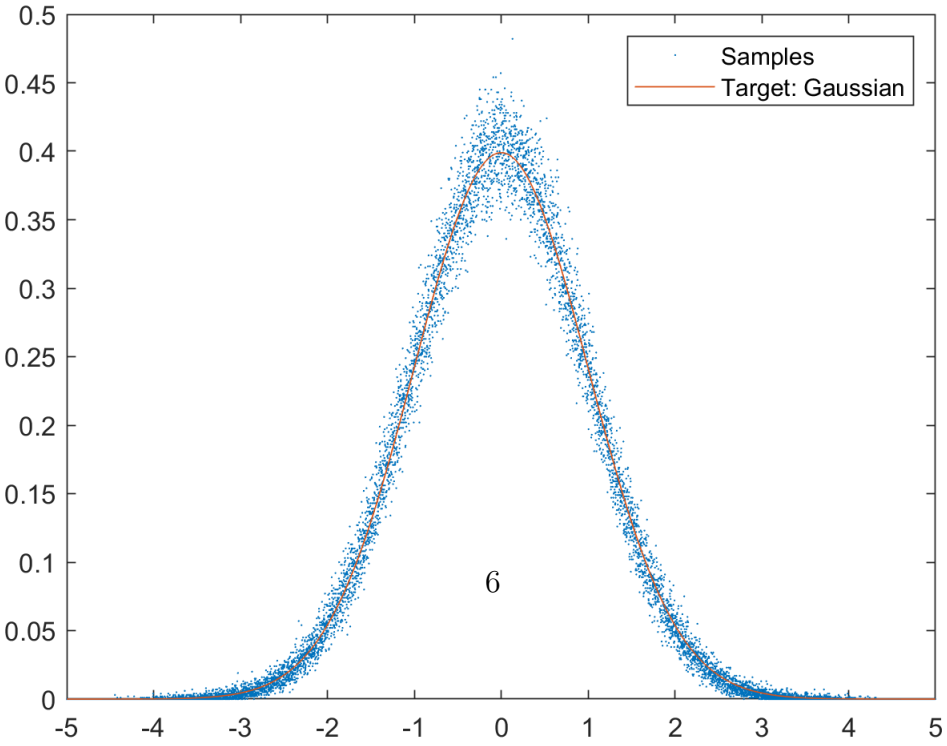


Figure 2: 50 Thousand Samples; Acceptance Rate = 0.4224

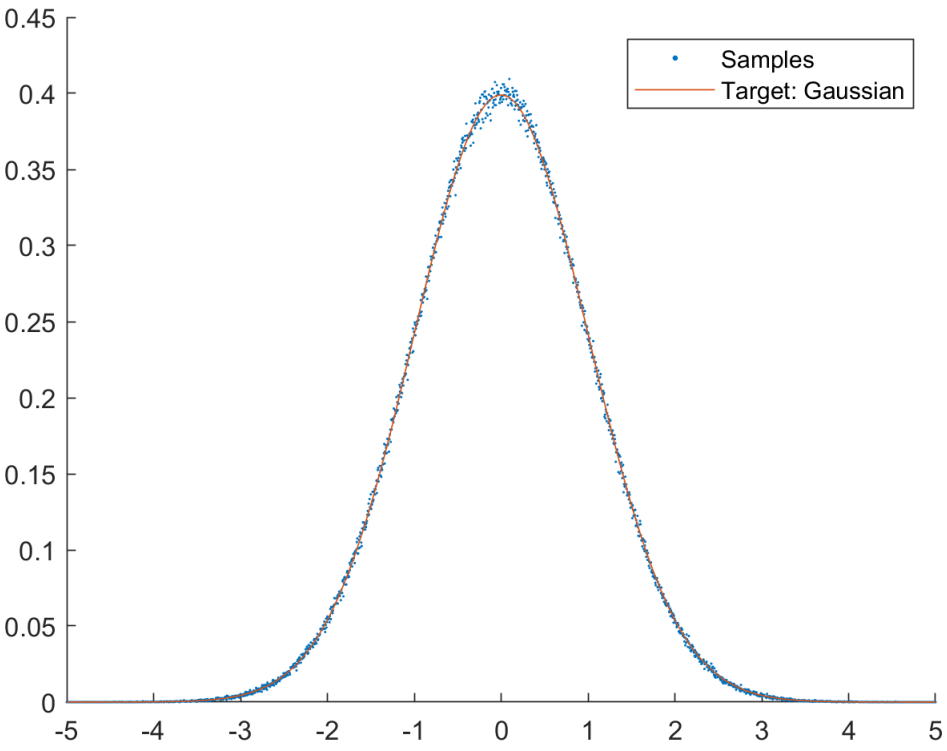


Figure 3: 5×10^6 Samples

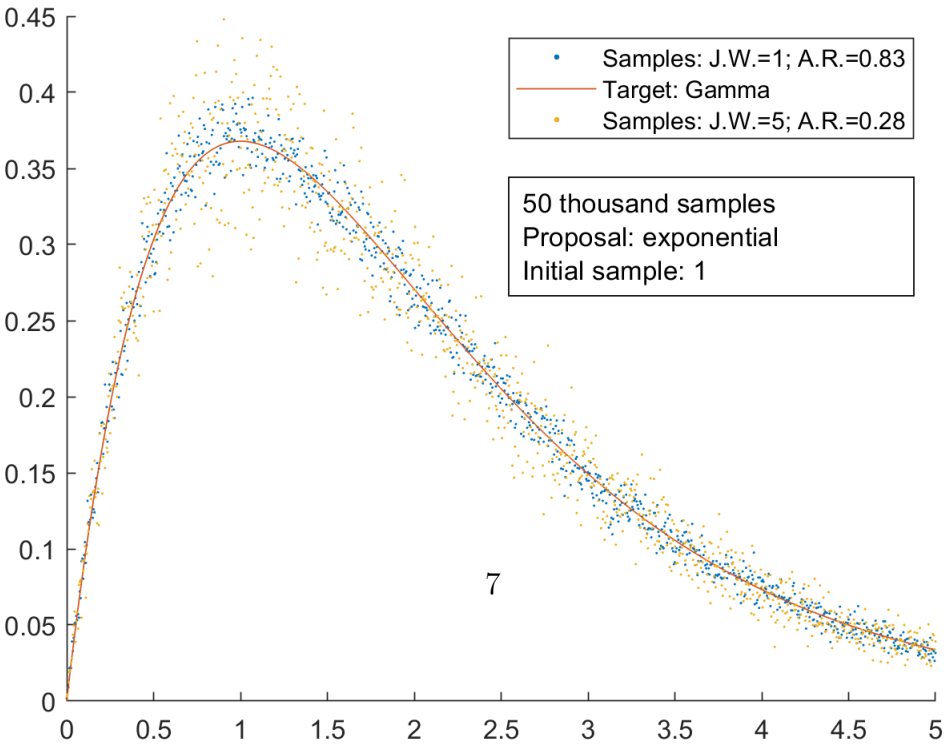


Figure 4: Sampling from Gamma with Exponential as Proposal

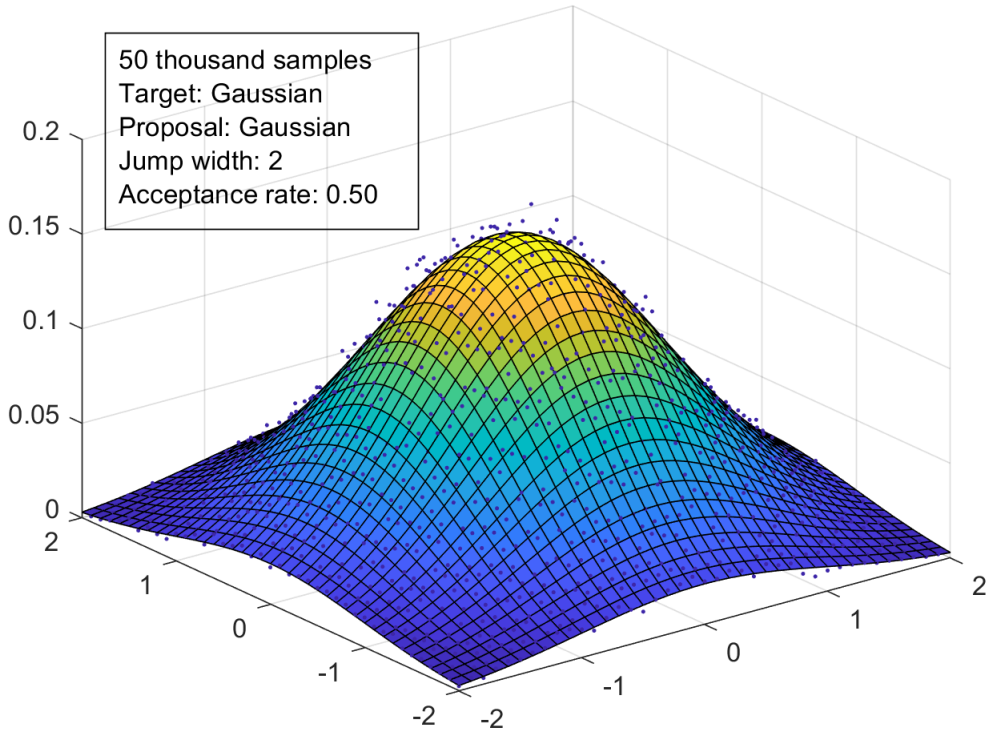


Figure 5: 2D-sampling by Gibbs sampling

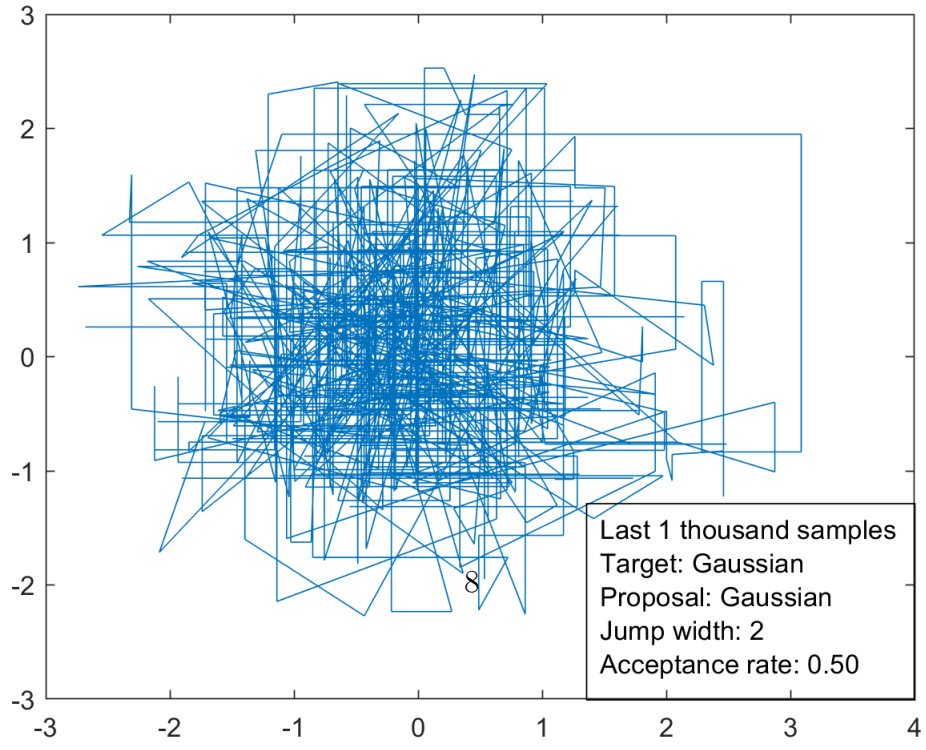


Figure 6: 2D random walk by Gibbs sampling

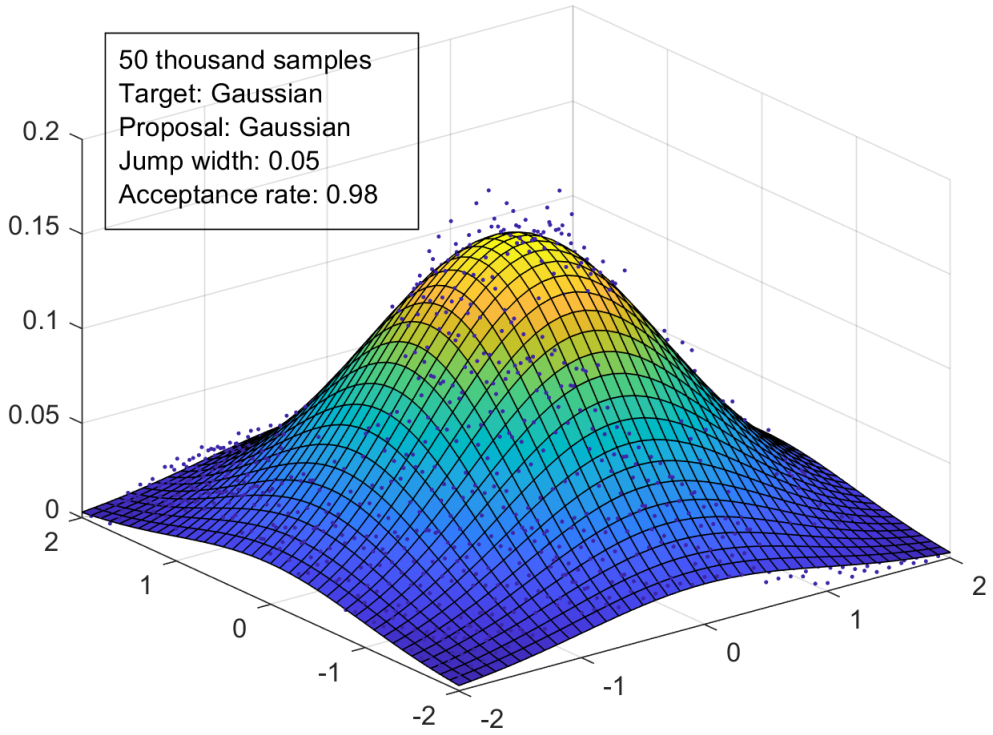


Figure 7: 2D-sampling by Gibbs sampling

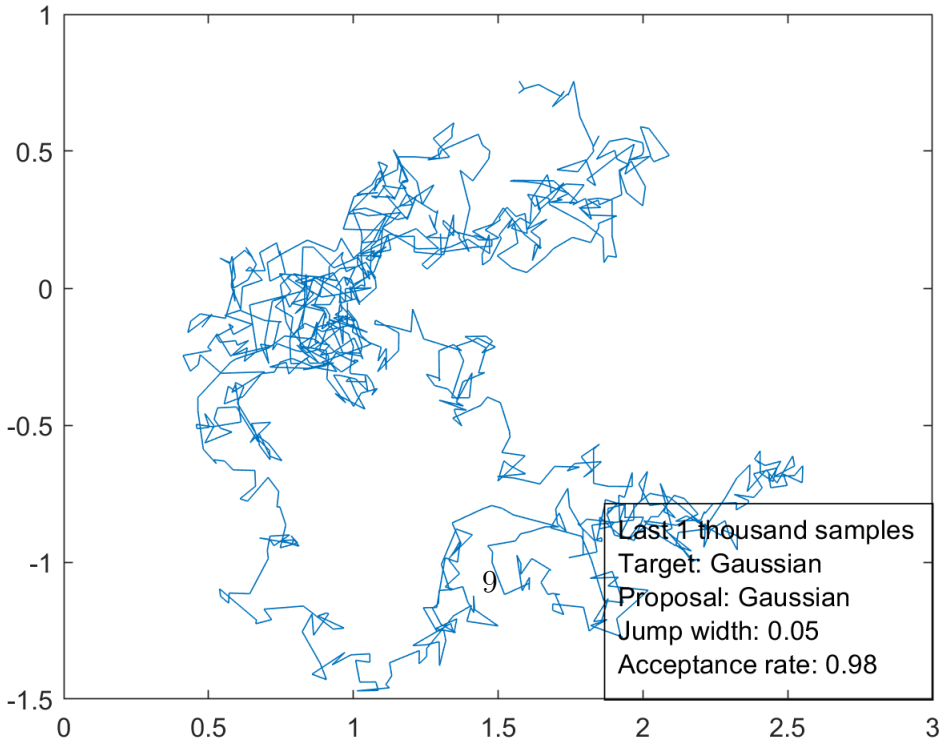


Figure 8: 2D random walk by Gibbs sampling

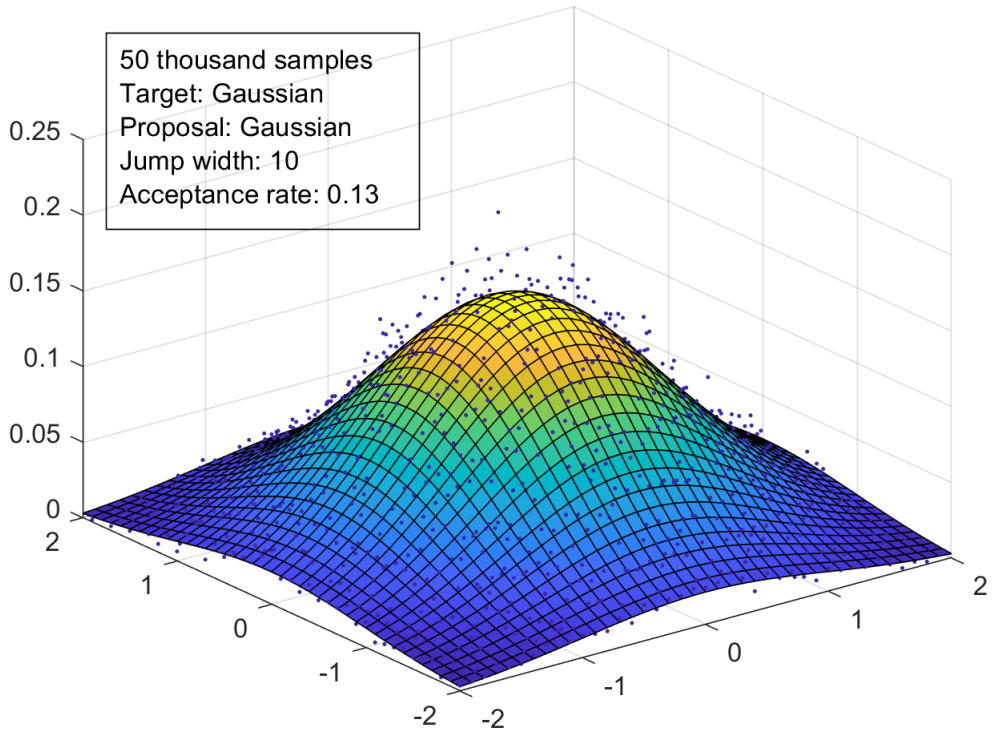


Figure 9: Gibbs Sampling: J.W. = 0.1; A.R. = 0.9740

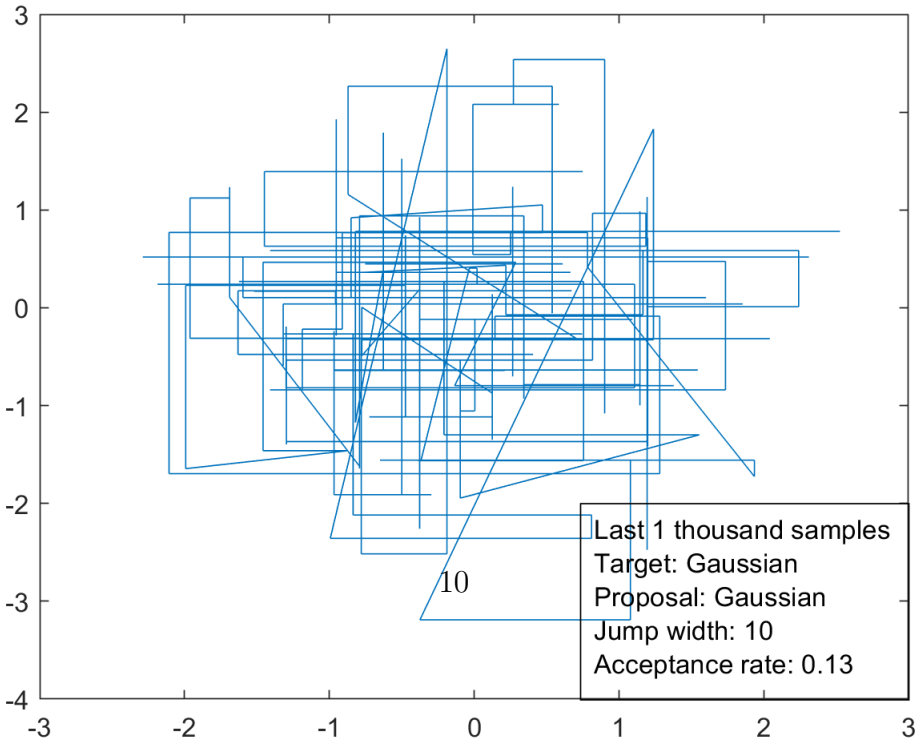


Figure 10: Gibbs Sampling: J.W. = 3; A.R. = 0.3720

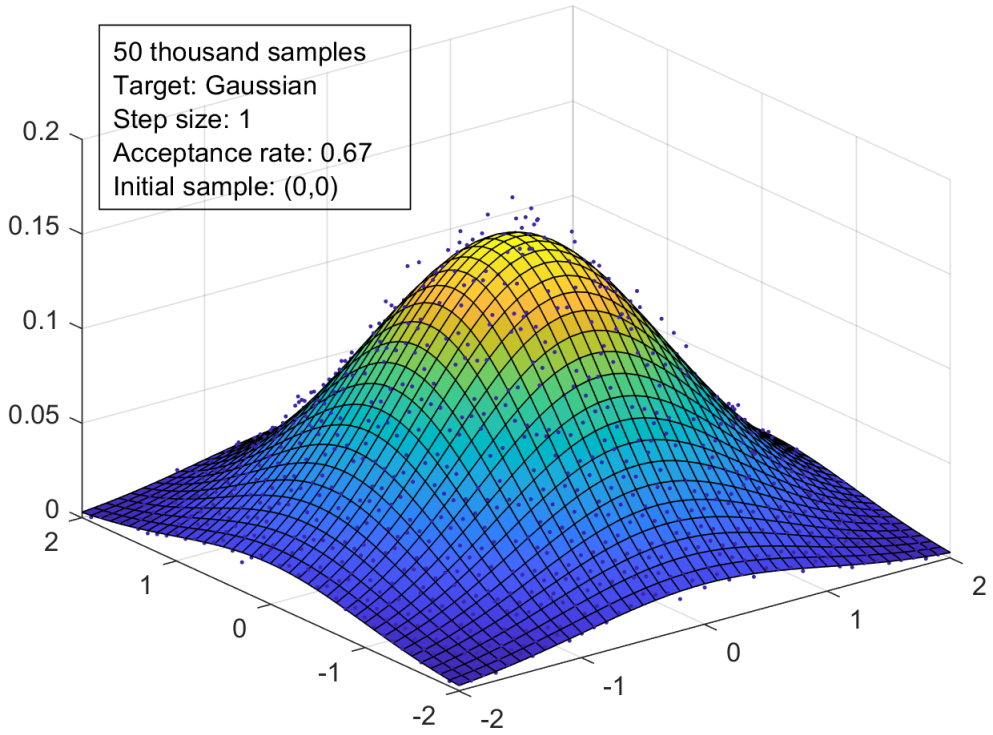


Figure 11: Metropolis-adjusted Langevin sampling

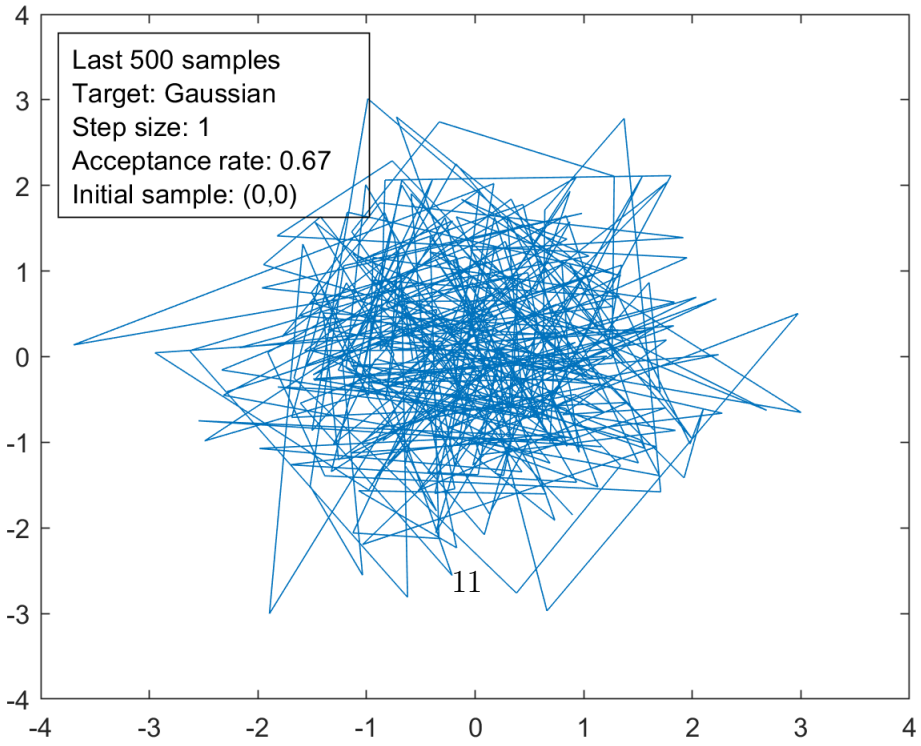


Figure 12: Metropolis-adjusted Langevin random walk

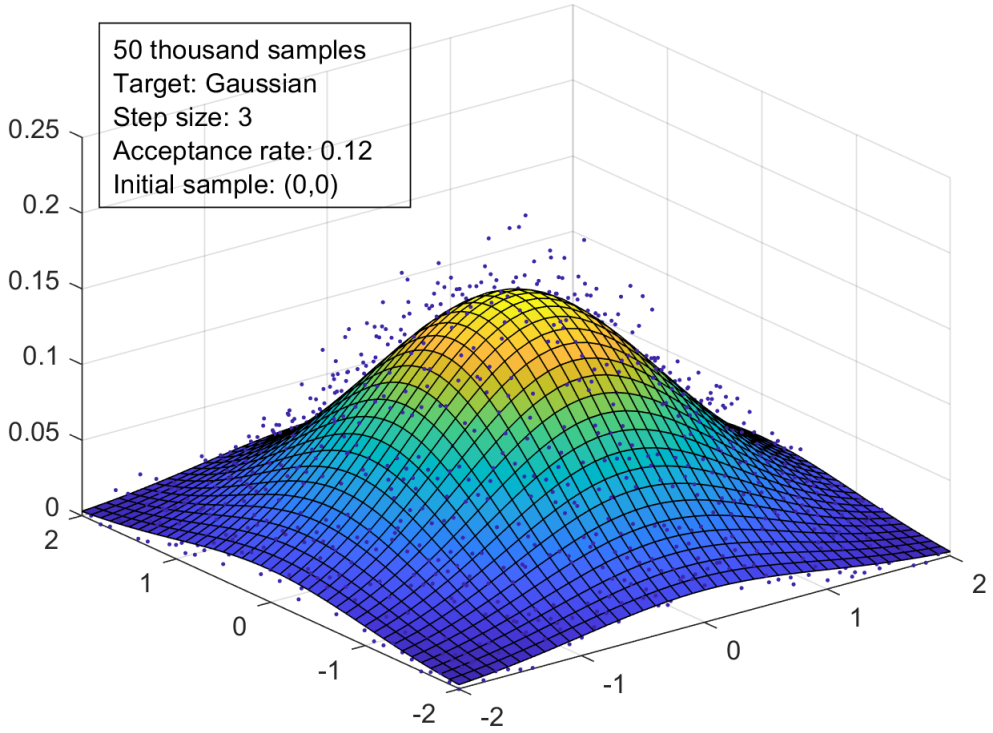


Figure 13: Metropolis-adjusted Langevin sampling

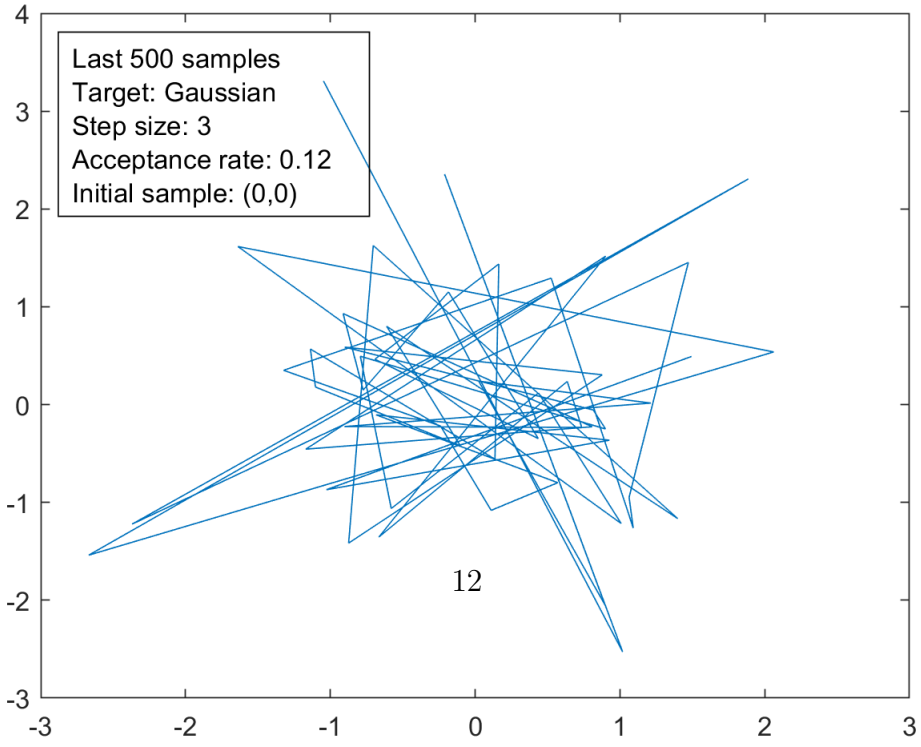


Figure 14: Metropolis-adjusted Langevin random walk

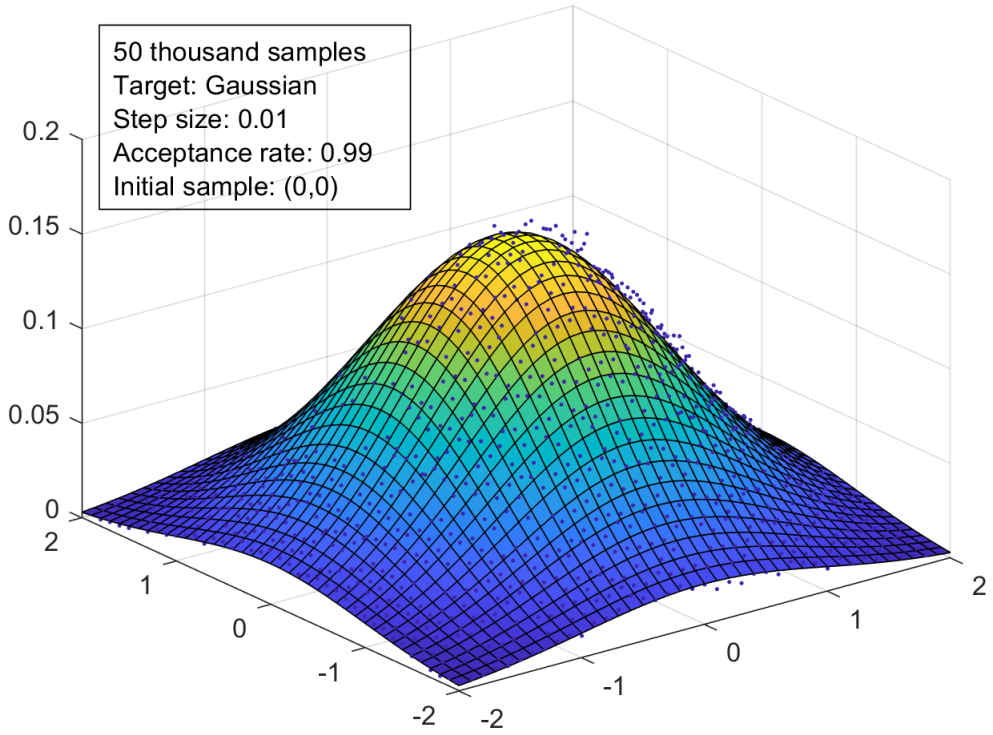


Figure 15: Metropolis-adjusted Langevin sampling

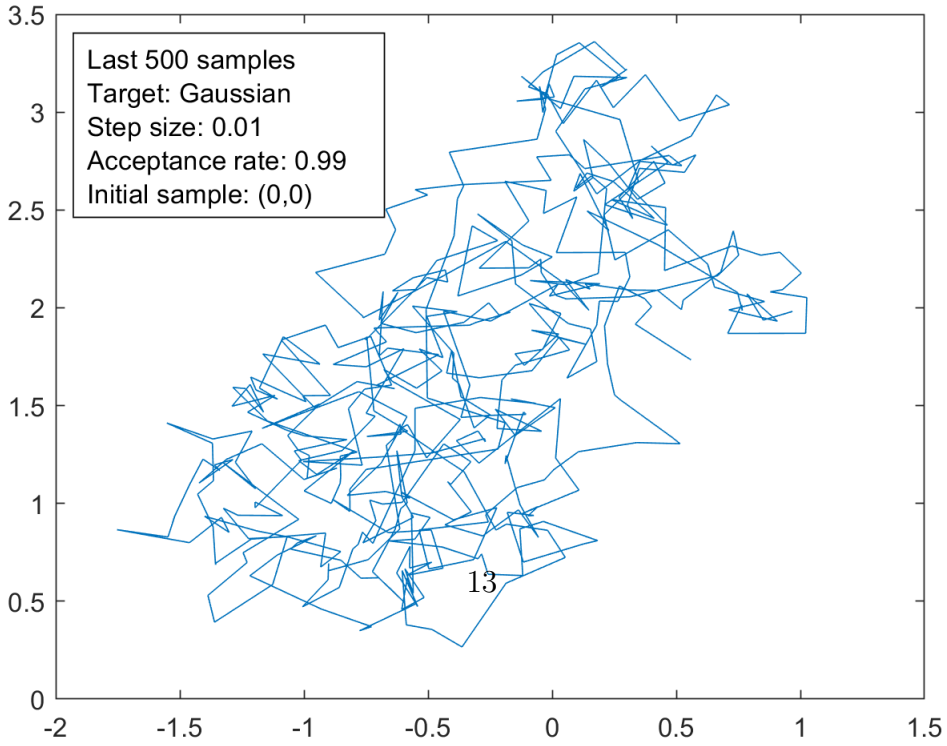


Figure 16: Metropolis-adjusted Langevin random walk

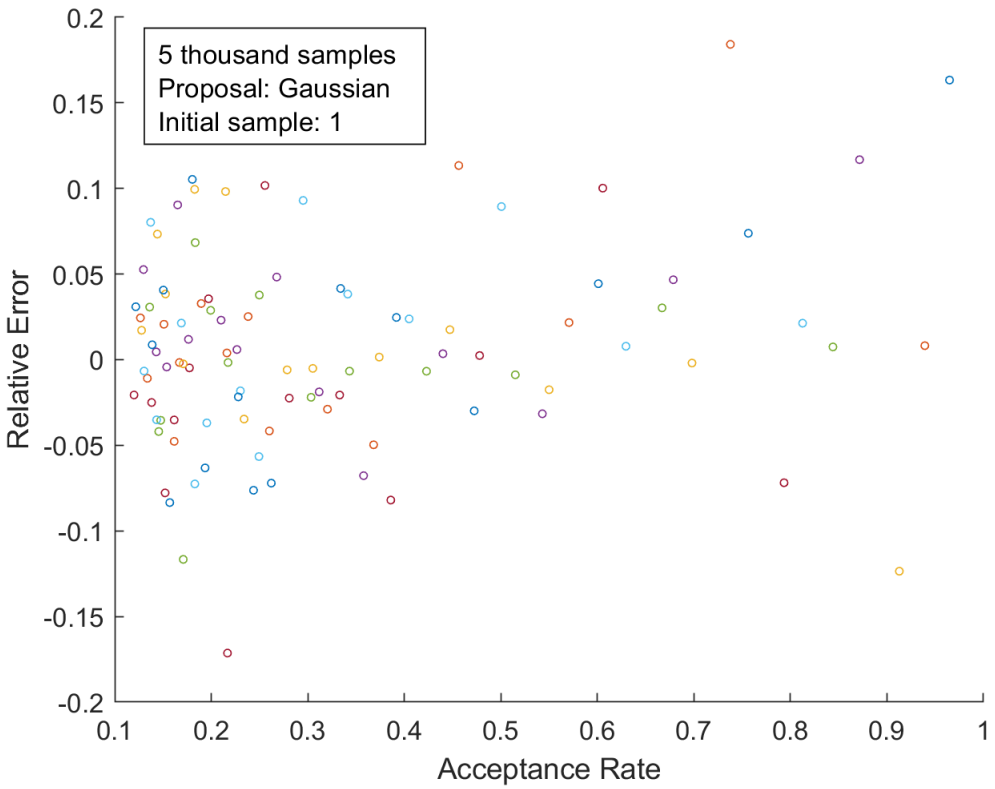


Figure 17: Relation between errors and acceptance rate

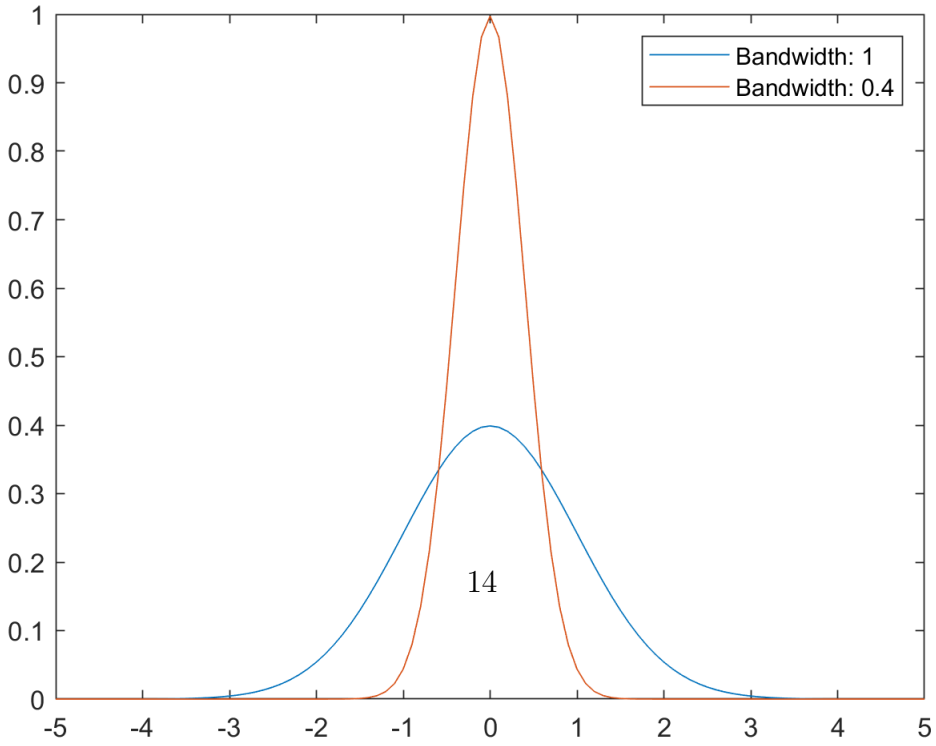


Figure 18: Guassian kernel and different bandwidths

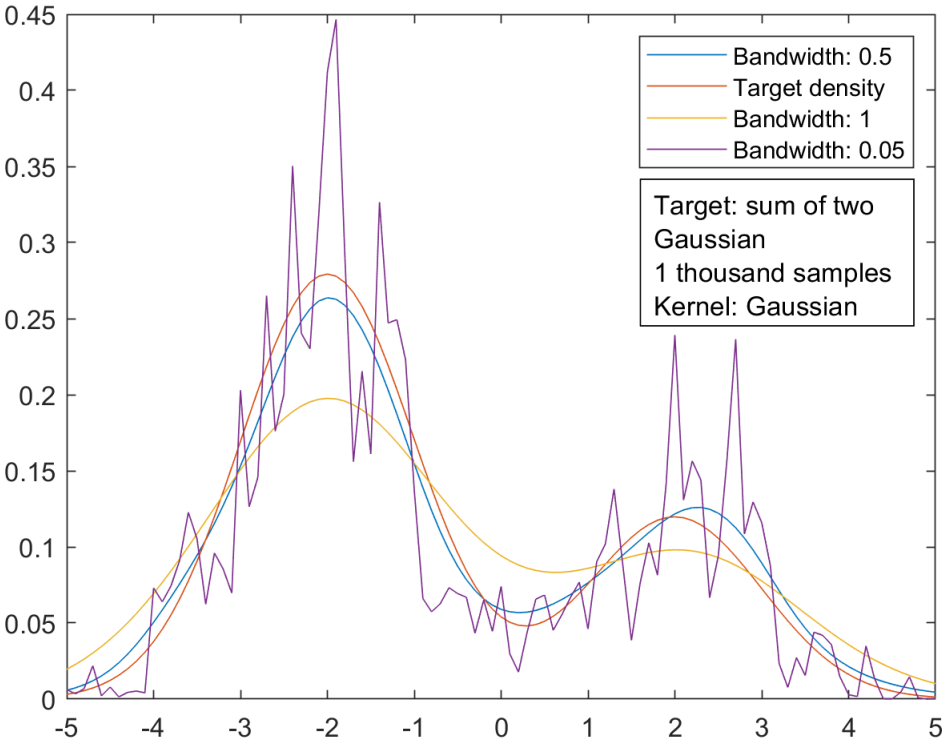


Figure 19: KDE

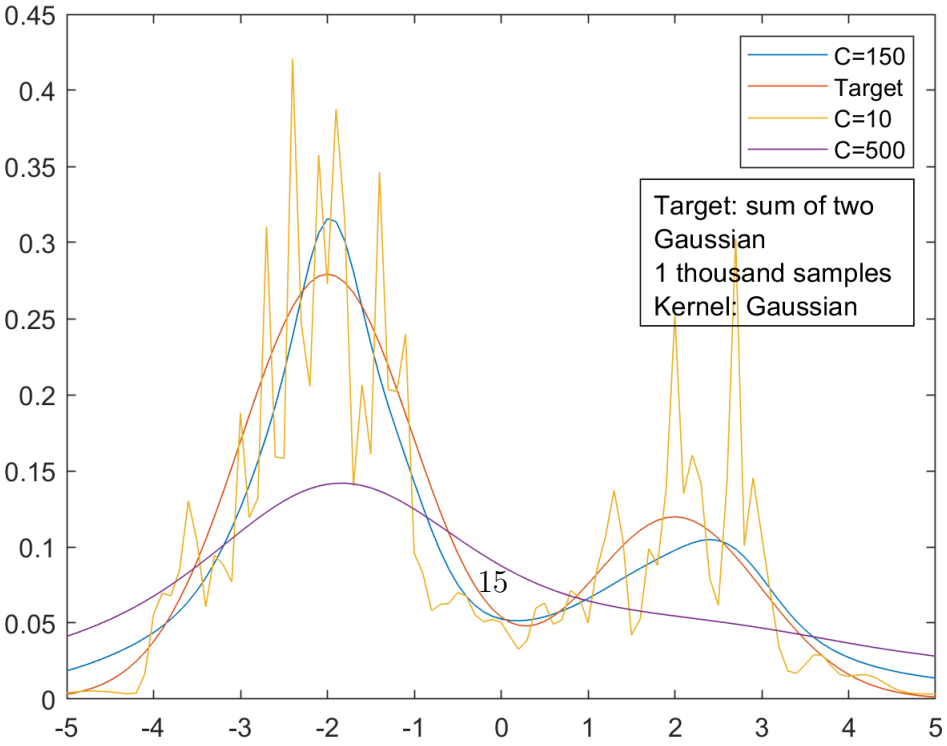


Figure 20: Variable KDE